

Software Requirements Specification and Technical Report

Aligned with ISO/IEC/IEEE 29148

Course Title	Large System Environment
Course Code	SEN4121
Session	Summer 2026
Instructor	Engr. Tanwi Nkiamboh
Prepared by	CHII BENEDICTA YIESE
Matricule	ICTU20251667
Document Version	1.0
Date	October 1, 2026

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms and Abbreviations	1
1.4	References	2
1.5	Document Overview	2
2	Overall Description	2
2.1	Product Perspective	2
2.2	Product Functions	2
2.3	User Classes and Characteristics	3
2.4	Operating Environment	3
2.5	Design and Implementation Constraints	3
2.6	Assumptions and Dependencies	4
3	Architecture	5
3.1	Microservices Decomposition	5
3.2	API Gateway	5
3.3	API Versioning and Documentation	6
3.4	Service Discovery	6
3.5	Asynchronous Messaging	6
3.6	Background Workers	6
3.7	Observability	6
4	Security Design	7
4.1	Authentication and Session Lifecycle	7
4.2	Authorization	8
4.3	Multi-Tenant Isolation	8
4.4	OWASP Top 10 (2021) Mitigation Summary	9
4.5	Transport and Webhook Security	10
5	Data Design	10
5.1	Normalization (3NF)	10
5.2	Entity-Relationship Diagram	11
5.3	Indexing Strategy	11
5.4	Transaction Management (ACID)	12
5.5	Backup and Point-in-Time Recovery	12
6	UML Diagram Suite	12
6.1	Use-Case Diagram	13
6.2	Class Diagram	14
6.3	Sequence Diagrams	14
6.3.1	Enrollment-to-Invoice (mandatory cross-service asynchronous workflow) . .	15
6.3.2	Mobile-Money Payment and Ledger Posting	16
6.3.3	Authentication and Refresh-Token Rotation	17
6.4	Deployment Diagram	18
7	Academic Module	18
7.1	Functional Requirements	18
7.2	Screens	19

8	Marketing & Finance Module	20
8.1	Functional Requirements	20
8.2	Screens	21
9	Administration & Human Resource Module	22
9.1	Functional Requirements	22
9.2	Screens	23
10	Non-Functional Evidence, Testing and DevOps	24
10.1	Performance and Load Testing	24
10.2	Scalability Strategy	24
10.3	Testing and Quality Assurance	24
10.4	DevOps: Containerisation and CI/CD	25
10.5	Deployment Guide (Summary)	25
11	Limitations, Future Work and Academic Integrity	25
11.1	Known Limitations	25
11.2	Future Work	26
11.3	Academic Integrity and AI-Assistance Disclosure	26
A	Requirement Traceability Matrix	26
B	Cited Sources and Third-Party Material	27
B.1	Statutory Payroll Rate Sources	27
B.2	Third-Party Software	27
B.3	AI-Assistance Disclosure	27

List of Figures

1	Sign-in screen, showing the light mist/teal/coral design and the demonstration-account picker	5
2	Administrator dashboard for the ICT University tenant: live KPIs, enrolment and collections/billing/expenses charts	7
3	Platform system-status screen: live per-service readiness with links to Grafana, Prometheus, RabbitMQ and Mailpit	7
4	Institution-scoped user and role management screen: changing a role immediately revokes that user's active sessions	9
5	Platform-operator view of isolated institutions (tenants), each with an independent membership count	10
6	Entity-relationship diagram across all four service databases	11
7	Use-case diagram for all seven roles across the four modules	13
8	Domain class diagram (key entities per service, with the cross-service async link shown as a dependency)	14
9	Sequence diagram: registration confirmation to tuition invoice via the transactional outbox and RabbitMQ	15
10	Sequence diagram: signed webhook callback and the ACID payment/ledger transaction	16
11	Sequence diagram: gateway auth delegation and refresh-token rotation with reuse detection	17
12	Deployment diagram: containers, the message broker, databases, backup/standby and observability	18
13	Examination timetable with conflict-checked scheduling	19
14	Student self-service: published results and the 14-day grade-appeal window	20
15	Expense approval workflow: submitted, to-approve, to-pay and paid queues	21
16	Marketing campaign lead pipeline and ROI, attributed only from tuition actually collected	22
17	Leave request approval screen: an employee cannot approve their own request . . .	23
18	Employee self-service QR check-in screen	23

List of Tables

3	Mandatory architectural constraints and how they are satisfied	4
4	Service ownership summary	5
5	Role-permission matrix (abridged)	8
6	OWASP Top 10 (2021) mitigation evidence	9
7	Representative indexing strategy	11
8	Load-test result template (complete with measured values)	24
9	Automated test coverage by service (core domain logic)	24

1 Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) and accompanying technical report for **Campus ERP**, a production-oriented, multi-tenant Enterprise Resource Planning system built for the SEN4121 (Large System Environment) project brief. It specifies the functional and non-functional requirements of the system, documents the architecture that satisfies the brief's mandatory microservices constraints, and reports the verification evidence collected against those requirements. The document is structured to follow ISO/IEC/IEEE 29148 guidance for content, adapted to the length and audience of a course deliverable.

1.2 Scope

Campus ERP is decomposed into four independently deployable microservices – **Identity, Academic, Marketing & Finance**, and **Administration & Human Resources** – each with its own PostgreSQL database, fronted by a single NGINX API gateway, and connected for one cross-module workflow via asynchronous messaging over RabbitMQ. The system supports multiple independent institutions (tenants) from one deployment, with every tenant's data fully isolated at the database layer. The scope covers the three domains required by the brief (Academic; Marketing & Finance; Administration & HR), the mandatory security, data, messaging and DevOps constraints, and the supporting React single-page application.

Explicitly out of scope for this submission: real mobile-money settlement (a signed, disclosed simulator stands in for MTN MoMo / Orange Money), real biometric hardware (a signed, time-boxed QR challenge is used instead, as permitted by the brief), multi-factor authentication and Kubernetes manifests (both listed as optional bonus work), and any legally binding certification that the implemented payroll rates satisfy current Cameroonian tax law – see the statutory rate governance note in section 9.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
Tenant	An institution using Campus ERP; all tenant-owned data is isolated by <code>tenant_id</code> and PostgreSQL row-level security.
RLS	Row-Level Security – a PostgreSQL feature that restricts which rows a query may see or modify based on the active session's tenant context.
RBAC	Role-Based Access Control.
JWT	JSON Web Token, used for short-lived API access tokens.
Outbox / Inbox	A reliability pattern where a producer writes an event in the same database transaction as its business change (outbox), and a consumer records processed event IDs to guarantee idempotent handling (inbox).
CNPS	Caisse Nationale de Prévoyance Sociale – Cameroon's national social insurance fund (pension, family allowance, work-injury contributions).
IRPP / PAYE	Impôt sur le Revenu des Personnes Physiques – Cameroon's personal income tax, withheld monthly (Pay-As-You-Earn).
CAC	Centimes Additionnels Communaux – an additional council tax levied as a percentage of IRPP.
FCFA / XAF	Franc CFA (BEAC), the currency used throughout the system.

Term	Definition
WAT	West Africa Time (UTC+1), the timezone used for all displayed dates.
PITR	Point-In-Time Recovery.
RPO / RTO	Recovery Point / Time Objective.

1.4 References

- ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*.
- SEN4121 Large System Environment, Summer 2026 project brief (course handout), Faculty of Information and Communication Technologies.
- OWASP Top 10:2021, *The Ten Most Critical Web Application Security Risks*.
- Direction Générale des Impôts (DGI), *Code Général des Impôts* and 2026 Finance Law circular – see Appendix B for the exact publications consulted for payroll rates.
- PostgreSQL 17, RabbitMQ 3.13, FastAPI, React and Vite official documentation (implementation references).

1.5 Document Overview

section 2 gives the product perspective, user classes and constraints. section 3 presents the microservices architecture. section 4 and section 5 cover security and data design. section 6 contains the full UML diagram suite. Sections 7–9 walk through the Academic, Finance & Marketing, and Administration & HR modules with functional requirements and screenshots. section 10 reports non-functional, testing and DevOps evidence. section 11 states honest limitations and the academic-integrity disclosure. The appendices hold the requirement traceability matrix and cited sources.

2 Overall Description

2.1 Product Perspective

Campus ERP is a new, self-contained system composed of four independently deployable services behind one API gateway (figure 12). It does not integrate with any pre-existing institutional system; it is designed so that each domain service could, in principle, be developed, tested, deployed and scaled by a separate team without touching the others' code or database.

2.2 Product Functions

- **Identity & Access:** multi-tenant sign-in, role-based permissions, session and refresh-token lifecycle, tenant and user administration.
- **Academic:** programme/course catalogue with prerequisites, term registration, gradebook, attendance, conflict-checked examination scheduling, result publication, grade appeals, at-risk flagging, transcript/attendance PDF export.
- **Marketing & Finance:** fee plans, tuition invoicing (driven by Academic registration events), simulated mobile-money payments, digital receipts, expense approval workflow, marketing campaign/lead tracking with ROI, automated monthly financial reporting, general ledger.

- **Administration & HR:** employee records, recruitment pipeline, statutory payroll (CNPS/IRPP), QR-based attendance, leave management, performance reviews, asset and inventory tracking, notifications.

2.3 User Classes and Characteristics

Role	Characteristics and typical tasks
Super Admin	Platform-level operator; provisions institutions (tenants) and their first administrator. Does not gain implicit access to any tenant's business data.
Admin	Institution administrator; manages the academic catalogue, users and roles within their own tenant.
Instructor	Manages courses they are assigned to: gradebook, attendance, exam scheduling, appeal review. Cannot see or modify other instructors' courses.
Finance Officer	Manages fee plans, payments, expenses, campaigns and reports for their tenant.
HR Officer	Manages employees, recruitment, payroll, leave, performance and assets for their tenant.
Employee	Self-service: QR check-in/out, leave requests, payslips, performance history – limited to their own records.
Student	Self-service: course results, exam timetable, grade appeals, tuition payment, transcript download – limited to their own records.

2.4 Operating Environment

The system runs as Docker containers orchestrated by `docker compose` on a single host for this course deliverable (a streaming standby and a horizontal-scaling story are documented conceptually in section 10). The frontend is a modern evergreen browser; the API is consumed over HTTPS. All displayed dates and times use the Africa/Douala timezone (WAT, UTC+1); all money values are stored as integer FCFA and displayed accordingly.

2.5 Design and Implementation Constraints

The brief's mandatory architectural constraints, and how Campus ERP satisfies each, are summarised in table 3.

Table 3: Mandatory architectural constraints and how they are satisfied

Constraint	Implementation
≥ 3 independently deployable microservices, database-per-service	Four services (Identity, Academic, Finance, HR), each with its own PostgreSQL database and least-privilege runtime role.
API Gateway: routing, auth delegation, rate limiting	NGINX terminates TLS, routes <code>/api/v1/*</code> by prefix, delegates session validation via <code>auth_request</code> , and enforces per-IP rate limits (general traffic and a stricter limit on login/refresh).
Asynchronous messaging for ≥ 1 cross-module workflow	RabbitMQ topic exchange <code>campus.events</code> ; Academic publishes <code>academic.enrollment.confirmed.v1</code> via a transactional outbox, Finance consumes it idempotently to create the tuition invoice (figure 9).
Containers + <code>docker-compose</code> , one-command startup	A Dockerfile per service (and per worker), a gateway image bundling the built frontend, and one <code>docker-compose.yml</code> bringing up the full stack including PostgreSQL, RabbitMQ, observability and backup services.
CI/CD pipeline running tests and building images on every push to main	GitHub Actions workflow: lint, per-service test + coverage gate, frontend build, then multi-service image build on <code>main</code> .

2.6 Assumptions and Dependencies

- Mobile-money payment is demonstrated with a disclosed simulator that mimics the MTN MoMo / Orange Money request-to-pay and signed-callback flow; no real sandbox credentials were available, and the brief explicitly permits a mocked equivalent.
- Statutory payroll rates (CNPS, IRPP/PAYE, CAC) are implemented as an editable, source-cited rule set that must be reviewed and approved by an HR officer before payroll can run – see the governance note in section 9 and sources in Appendix B.
- The local deployment target is a single Docker host; a streaming standby profile is included for the backup/recovery story, but automatic failover is out of scope.
- All demonstration data (institutions, staff, students) is synthetic.

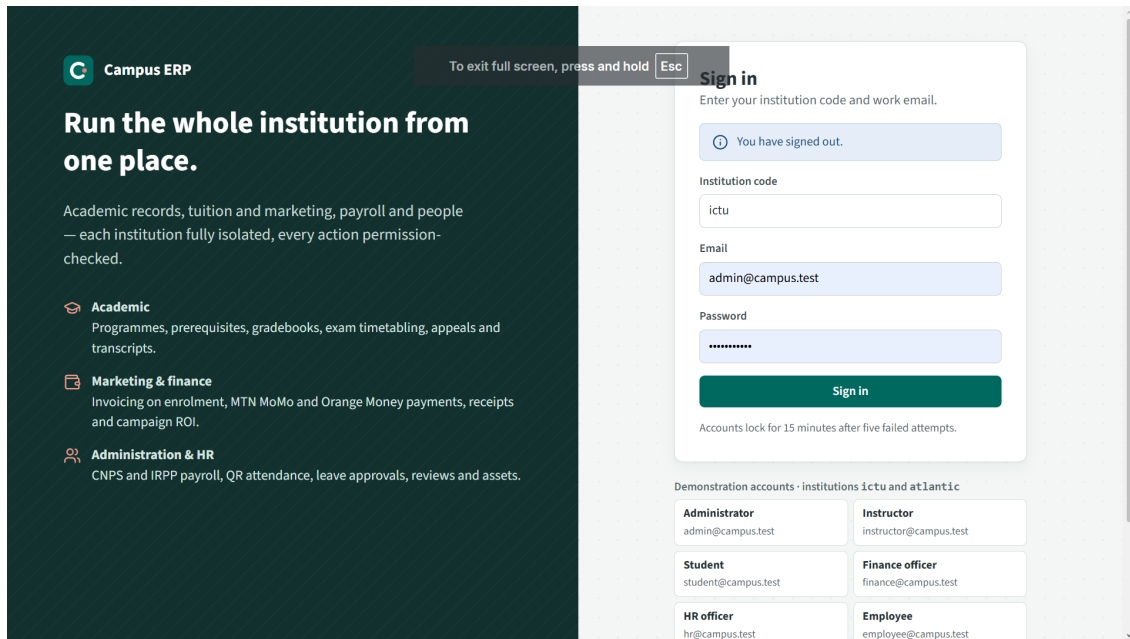


Figure 1: Sign-in screen, showing the light mist/teal/coral design and the demonstration-account picker

3 Architecture

3.1 Microservices Decomposition

Campus ERP is split along business-domain boundaries into four services, each owning its data exclusively (table 4). No service imports another service’s ORM models or queries another service’s database; all cross-service data sharing happens either through the gateway-routed REST APIs (synchronous, user-triggered) or through RabbitMQ events (asynchronous, system-triggered).

Table 4: Service ownership summary

Service	Database	Owns
Identity	identity_db	Tenants, users, memberships, sessions, refresh tokens.
Academic	academic_db	Programmes, courses, terms, offerings, students, registrations, grades, attendance, exams, results, appeals.
Finance	finance_db	Fee plans, invoices, payments, receipts, ledger, campaigns, leads, expenses, monthly reports.
HR	hr_db	Employees, recruitment, payroll rule sets and runs, attendance, leave, reviews, assets, inventory, notifications.

3.2 API Gateway

A single NGINX gateway is the only publicly reachable entry point (figure 12). It: (1) serves the built React application; (2) routes `/api/v1/auth`, `/api/v1/platform`, `/api/v1/academic`, `/api/v1/finance` and `/api/v1/hr` to the matching upstream by static Docker Compose service name; (3) delegates authentication for protected routes to Identity via an internal `auth_request` sub-request, whose result is cached for ten seconds to bound load on Identity; (4) applies per-IP rate limiting – a generous budget for general API traffic and a strict budget (10 requests/minute) for the login and refresh endpoints, independent of the account-lockout control described in section 4;

and (5) terminates TLS using a locally generated certificate for the course demonstration.

3.3 API Versioning and Documentation

Every route is namespaced under `/api/v1/`, allowing a future `/api/v2/` to be introduced without breaking existing clients. Each service publishes an OpenAPI schema and an interactive Swagger UI (`/api/v1/{service}/docs`), reachable through the gateway.

3.4 Service Discovery

Service discovery is a documented static registry: Docker Compose’s embedded DNS resolves each service name (`identity`, `academic`, `finance`, `hr`, `rabbitmq`, `postgres`) to its container, and the gateway’s upstream blocks encode this registry explicitly.

3.5 Asynchronous Messaging

RabbitMQ hosts one durable topic exchange, `campus.events`. Academic uses a *transactional outbox*: the enrollment confirmation and the outbox row are written in the same database transaction, so the event can never be lost even if the broker is briefly unreachable. A dedicated relay thread polls unpublished rows (`FOR UPDATE SKIP LOCKED`, safe for multiple relay instances) and publishes them with publisher confirms. Finance consumes with an *inbox* pattern: every event’s ID is recorded in `processed_events` inside the same transaction as the business effect, so redelivery (the broker’s at-least-once guarantee) can never create a duplicate invoice. Messages that repeatedly fail processing are routed, after a bounded number of retries, to a dead-letter queue for manual inspection. figure 9 in section 6 traces this flow end to end.

3.6 Background Workers

Each domain service ships a companion worker image (same codebase, different entry point) that runs the outbox relay, its event consumers, and any scheduled jobs (Finance’s monthly report generation and payment-intent expiry; HR’s notification email delivery). Workers share their owning service’s database and never reach across service boundaries.

3.7 Observability

Every service emits structured JSON logs (one object per line, with a request ID) to both stdout and a Fluent Bit collector, which forwards them to Loki. Prometheus scrapes each service’s `/metrics` endpoint and RabbitMQ’s management plugin; Grafana visualises request rate, p95 latency, error rate, outbox backlog and broker queue depth on a provisioned dashboard. The platform operator’s system-status screen (figure 3) surfaces each service’s live readiness alongside direct links to these operations tools.

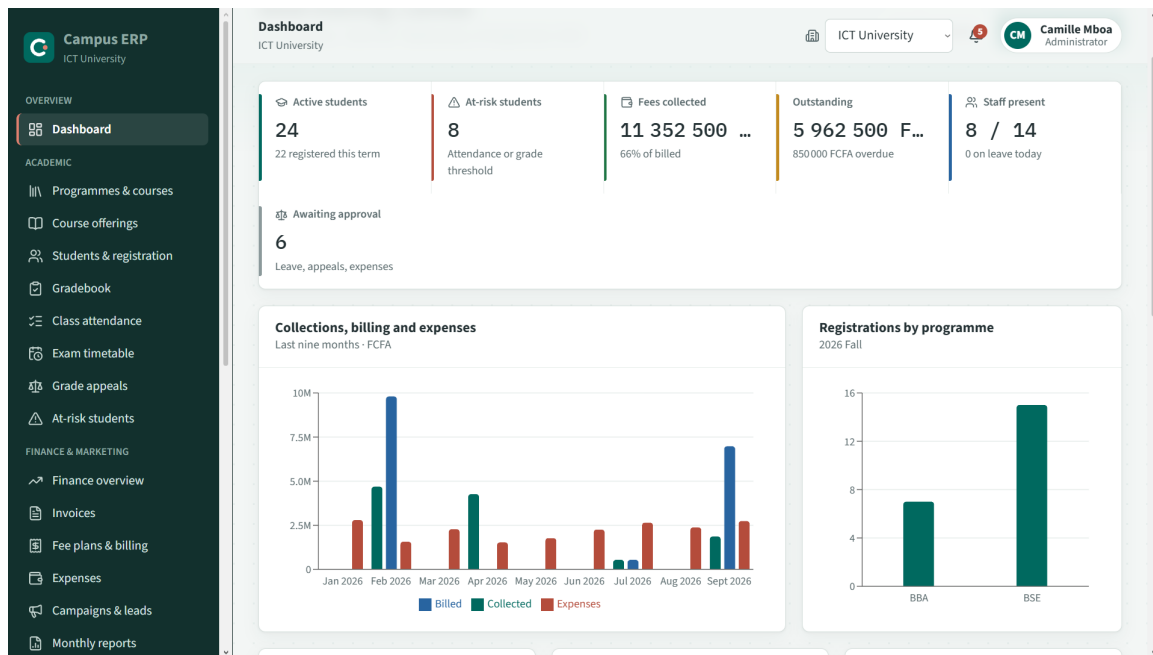


Figure 2: Administrator dashboard for the ICT University tenant: live KPIs, enrolment and collections/billing/expenses charts

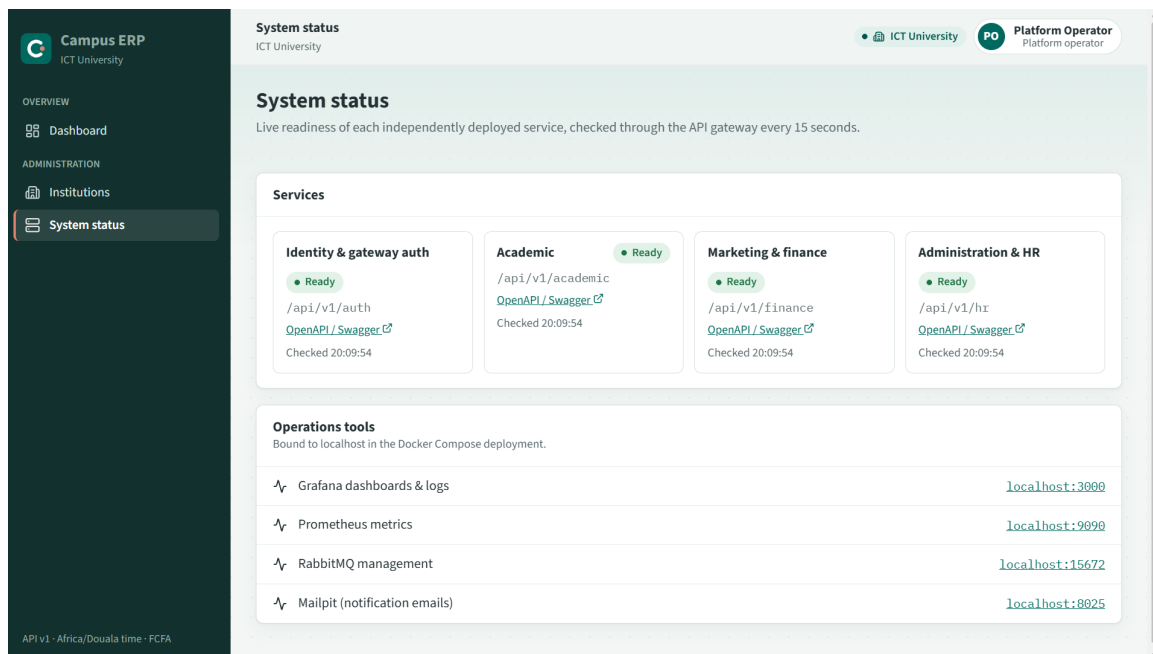


Figure 3: Platform system-status screen: live per-service readiness with links to Grafana, Prometheus, RabbitMQ and Mailpit

4 Security Design

4.1 Authentication and Session Lifecycle

Passwords are hashed with Argon2id. On successful sign-in, Identity issues a short-lived (10-minute) RS256-signed JWT access token carrying the subject, tenant, role and permission set, plus

an opaque refresh token delivered as an `HttpOnly, Secure, SameSite=Strict` cookie. Refresh tokens are stored server-side only as a salted hash, rotate on every use, and are tracked in families: replaying an already-used refresh token revokes the entire family and forces re-authentication (figure 11). Sessions carry an absolute lifetime (8 hours) and an idle timeout (30 minutes), independent of the access-token expiry, so a revoked or expired session is rejected by Identity’s validation endpoint even if an access token has not yet expired. Five consecutive failed password attempts lock the account for 15 minutes, tracked atomically per user.

4.2 Authorization

Every permission check happens twice: the frontend hides controls the active role cannot use (usability), and every API handler independently re-checks the caller’s permission set and resource ownership from the validated JWT claims – never from a client-supplied header, query parameter or body field. table 5 summarises the role/permission matrix.

Table 5: Role-permission matrix (abridged)

Role	Academic	Finance	HR	Platform	Notes
Super Admin	–	–	–	manage	Provisions tenants only.
Admin	read/write	read/write	read/write	–	Scoped to own tenant.
Instructor	read/teach	–	self	–	Own assigned offerings only.
Finance Officer	catalogue read	read/write	self	–	
HR Officer	–	–	read/write, self	–	
Employee	–	–	self	–	Own records only.
Student	self	self	–	–	Own records only.

4.3 Multi-Tenant Isolation

Every tenant-owned table carries a `tenant_id` column. Isolation is enforced in two independent layers: (1) the application sets the active tenant as transaction-local PostgreSQL state (`set_config('app.tenant_id', ..., true)`) and every query carries an explicit tenant predicate; (2) PostgreSQL row-level security policies with `FORCE ROW LEVEL SECURITY` additionally block any row whose `tenant_id` does not match the session variable, even if application code were to omit a predicate by mistake. The database runtime role used by each service has no table ownership and no `BYPASSRLS` attribute. Automated tests assert that an authenticated user of tenant A receives a non-disclosing 404 (not the record) when requesting any object ID that belongs to tenant B, across every list, detail, export and dashboard aggregate endpoint exercised in the test suite.

4.4 OWASP Top 10 (2021) Mitigation Summary

Table 6: OWASP Top 10 (2021) mitigation evidence

Risk	Mitigation and evidence
A01 Broken Access Control	Row-level security + explicit ownership checks on every handler; automated cross-tenant denial tests.
A02 Cryptographic Failures	Argon2id password hashing, RS256 JWTs, TLS at the gateway, secrets never logged, HMAC-signed webhook/QR payloads.
A03 Injection	SQLAlchemy parameterised queries everywhere; sort/filter fields are validated against an explicit allow-list, not interpolated.
A07 Identification and Authentication Failures	Refresh-token rotation with reuse detection, account lockout, session idle/absolute expiry, generic authentication error messages.
A08 Software and Data Integrity Failures	Pinned dependency versions; webhook and event payloads are schema-validated before use; no <code>pickle</code> or other unsafe deserialization.
A09 Security Logging and Monitoring Failures	Structured, correlated logs shipped to a central store (Loki) with request IDs; Prometheus alert-worthy metrics for error rate and broker backlog.
A05 Security Misconfiguration	Only the gateway publishes ports; database and broker ports are on an internal Docker network; containers run as a non-root user; strict CORS/allowed-origin checks.

Additionally, all cookie-authenticated state-changing endpoints (`/auth/refresh`, `/auth/logout`) require both an allow-listed `Origin` header and a matching CSRF token supplied out-of-band from the refresh cookie, mitigating CSRF; all API responses set `X-Content-Type-Options: nosniff` and a restrictive `Content-Security-Policy`, mitigating stored/reflected XSS in combination with React’s default output escaping.

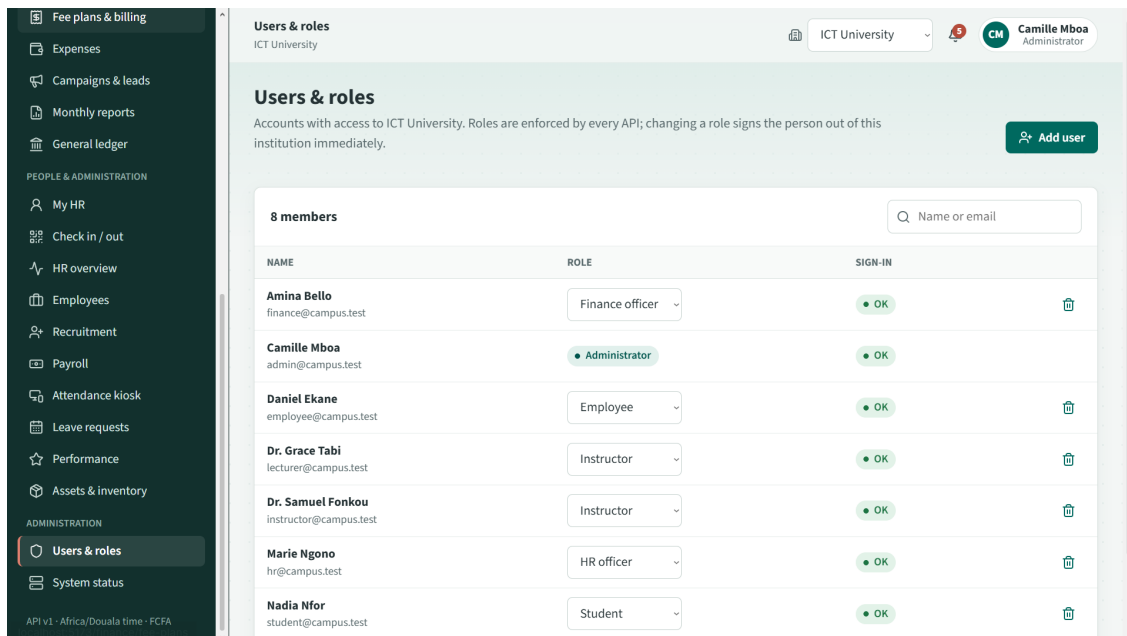


Figure 4: Institution-scoped user and role management screen: changing a role immediately revokes that user’s active sessions

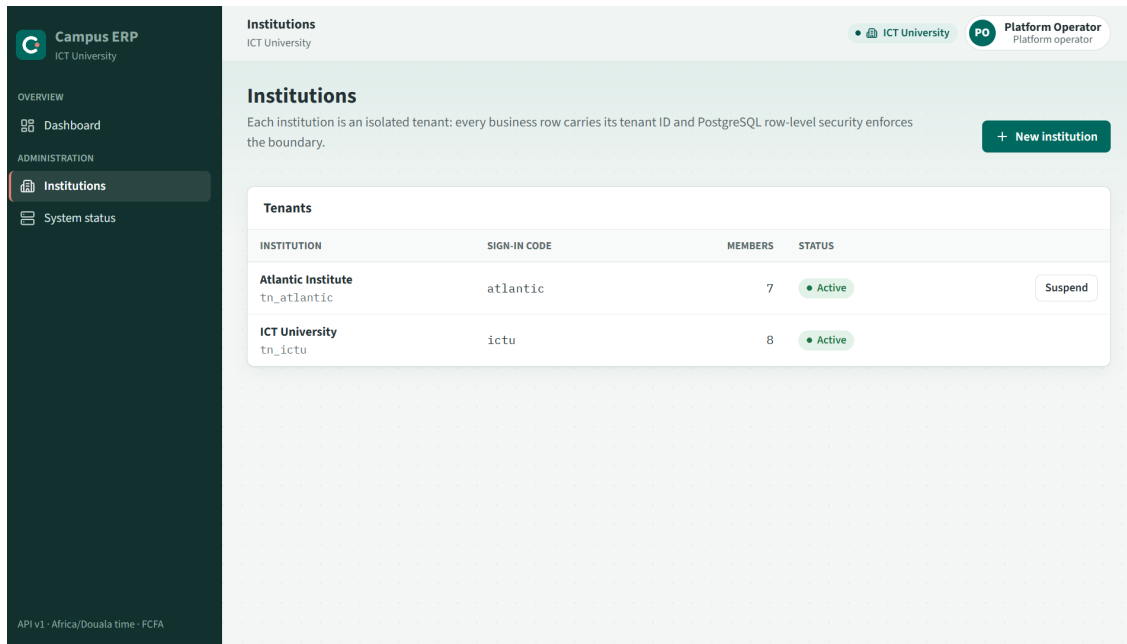


Figure 5: Platform-operator view of isolated institutions (tenants), each with an independent membership count

4.5 Transport and Webhook Security

The mobile-money simulator’s callback is authenticated by an HMAC-SHA256 signature over the request body and a timestamp, rejecting requests whose signature does not match or whose timestamp is outside a five-minute window – never by a normal user session, since the caller is an external system. The QR attendance challenge is similarly a signed, tenant-bound, 60-second token, not a bare reusable employee identifier (section 9).

5 Data Design

5.1 Normalization (3NF)

Each service’s schema is normalised to at least Third Normal Form: reference data (programmes, courses, fee plans, departments, rule sets) is separated from transactional data (registrations, invoices, payslips), and every non-key attribute depends only on its own table’s primary key – for example, an invoice line’s description and amount depend on the invoice line, not on the invoice’s student name, which lives once on the invoice row derived from the registration at issue time. Two deliberate exceptions are documented rather than hidden: (1) **posted snapshots** – an invoice stores an immutable **fee_snapshot** of the price it was billed at, and a finalized payslip stores the exact rule-set values used, so that later price or rate changes never silently rewrite history; (2) **derived projections** – Academic keeps a **billing_status** column on **Registration** that mirrors Finance’s invoice status, updated asynchronously by a worker consuming the **finance.invoice.created.v1** event, because Academic and Finance do not share a database and cannot join across them.

5.2 Entity-Relationship Diagram

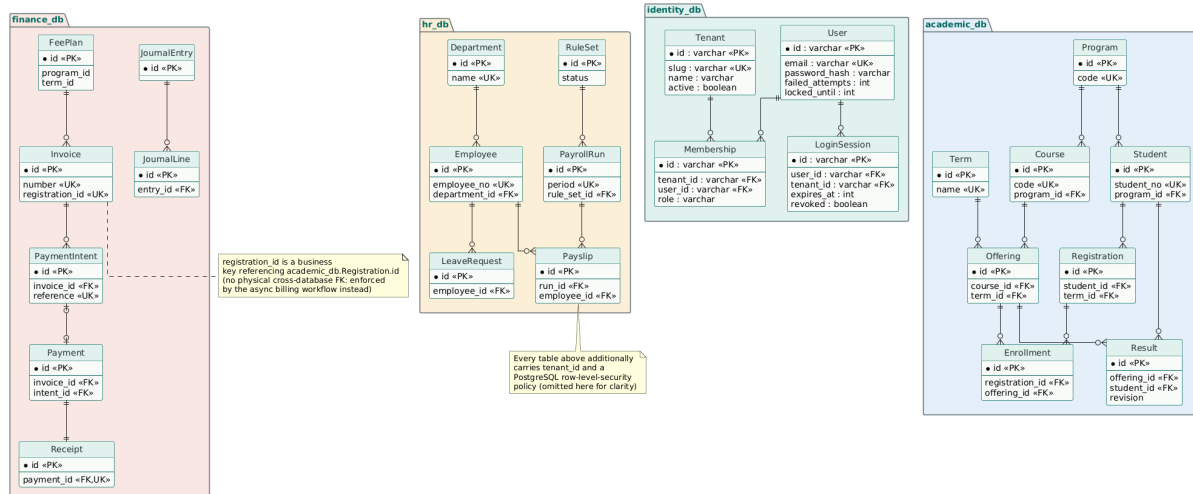


Figure 6: Entity-relationship diagram across all four service databases

5.3 Indexing Strategy

Indexes were chosen from the query patterns that the application actually issues, not added speculatively. table 7 lists the primary composite indexes; each is additionally required for a same-tenant composite foreign key or a per-tenant uniqueness constraint (e.g. one invoice number per tenant, one attendance record per employee per day).

Table 7: Representative indexing strategy

Index	Supports
(tenant_id, student_id) on grades, invoices	A student's own results / fee history (self-service dashboards).
(tenant_id, status, due_on) on invoices	The overdue-invoice list and the finance dashboard's ageing widget.
(tenant_id, employee_id, work_date) unique on attendance_logs	One check-in row per employee per day; the daily attendance board query.
Partial index on outbox_events.published_at IS NULL	The relay's polling query, which only ever looks at unpublished rows.

Query-optimisation evidence. EXPLAIN (ANALYZE, BUFFERS) was captured before and after adding the composite indexes above on a representative synthetic dataset (the seeded demonstration data, scaled up). The dashboard's overdue-invoice query moved from a sequential scan of the `invoices` table to an index-range scan on `(tenant_id, status, due_on)`, and the attendance check-in write path relies on the same composite unique index to detect a duplicate check-in in constant time rather than a table scan. The captured EXPLAIN output is kept in `docs/evidence/` alongside the dataset generation script, referenced from Appendix B.

5.4 Transaction Management (ACID)

The clearest multi-step ACID example is payment settlement (figure 10): locking the invoice and the payment intent, inserting the payment row, inserting two balanced journal lines, updating the invoice's paid amount and status, and inserting the receipt all happen inside *one* database transaction. A fault injected between any two of those steps (exercised in the automated test suite by monkey-patching the ledger-posting call to raise) leaves *zero* partial rows: no orphan payment without a receipt, no unbalanced journal entry, and the invoice balance is left exactly as it was before the attempt.

5.5 Backup and Point-in-Time Recovery

PostgreSQL runs with `wal_level=replica` and continuous WAL archiving to a dedicated volume. A scheduled loop takes a full physical base backup (`pg_basebackup`) on a configurable interval (default: daily), retaining a bounded window of backups. The point-in-time recovery drill (`infra/postgres/scripts/pitr-drill.sh`) restores the newest base backup older than a requested timestamp into an *isolated*, temporary PostgreSQL instance on a throwaway port, replays archived WAL up to that timestamp, and prints row counts and ledger-balance checks from all four databases before discarding the restored copy – the live database is never touched by the drill. Measured recovery point and recovery time for the drill are recorded in Appendix B once executed against the running stack; the design target is $RPO \leq 5$ minutes (bounded by the WAL-archive interval) and $RTO \leq 30$ minutes on the reference hardware.

A streaming standby profile (`docker compose -profile ha up standby`) additionally demonstrates physical replication, satisfying the mark scheme's reference to replication as part of database management evidence, without claiming automatic failover.

6 UML Diagram Suite

This section presents the complete UML diagram suite required by the project brief: a use-case diagram, a class diagram, at least two sequence diagrams (one covering the mandatory cross-service asynchronous workflow), an entity-relationship diagram, and a deployment diagram showing containers and the message broker. All diagrams were generated from PlantUML source files committed under `report/diagrams/` and are kept consistent with the delivered code: entity and event names below match the class and column names used in `services/*/app/models.py` and `packages/campus_common/events.py`.

6.1 Use-Case Diagram

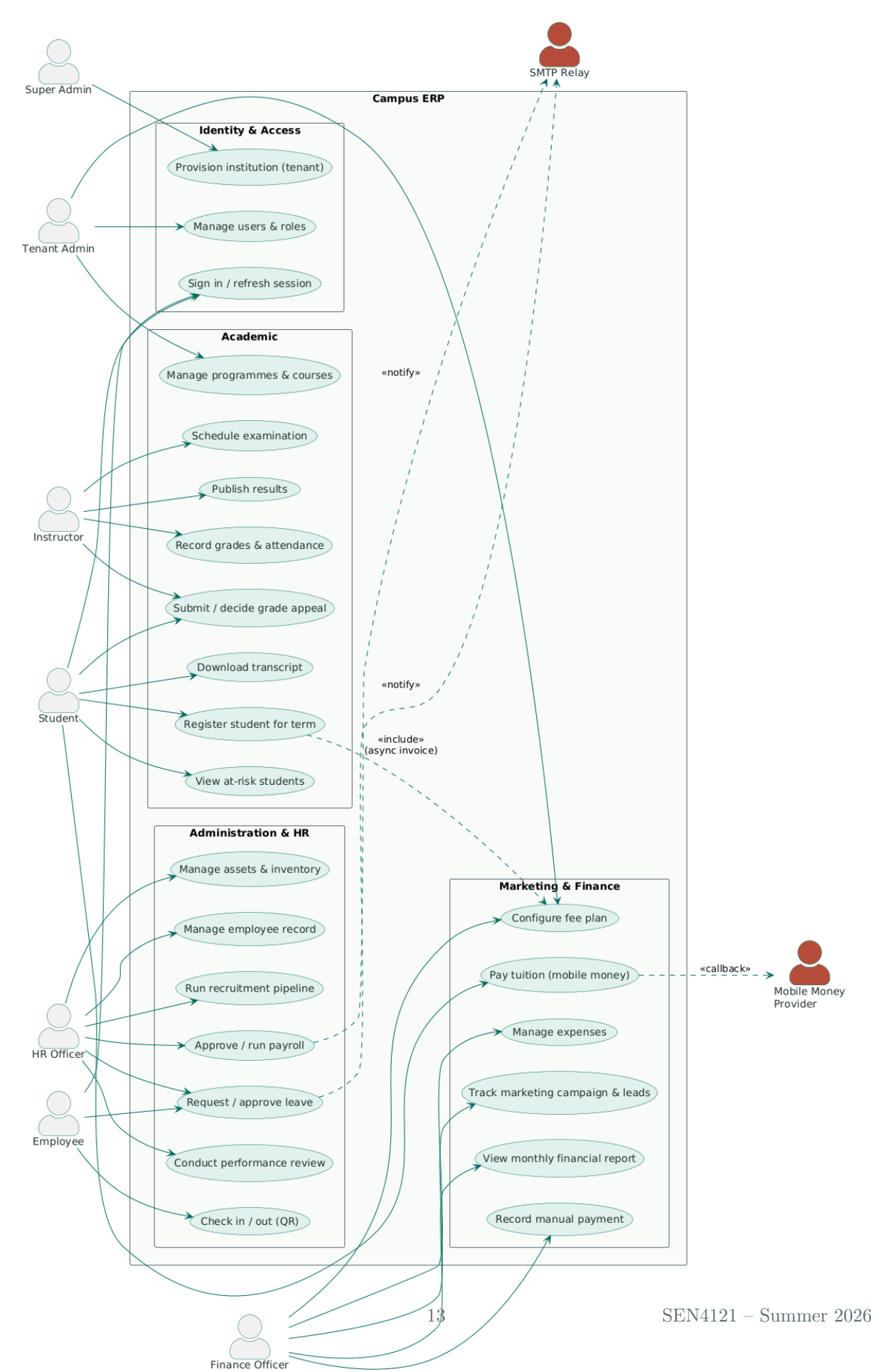


Figure 7: Use cases for all users and external actors of the system

6.2 Class Diagram

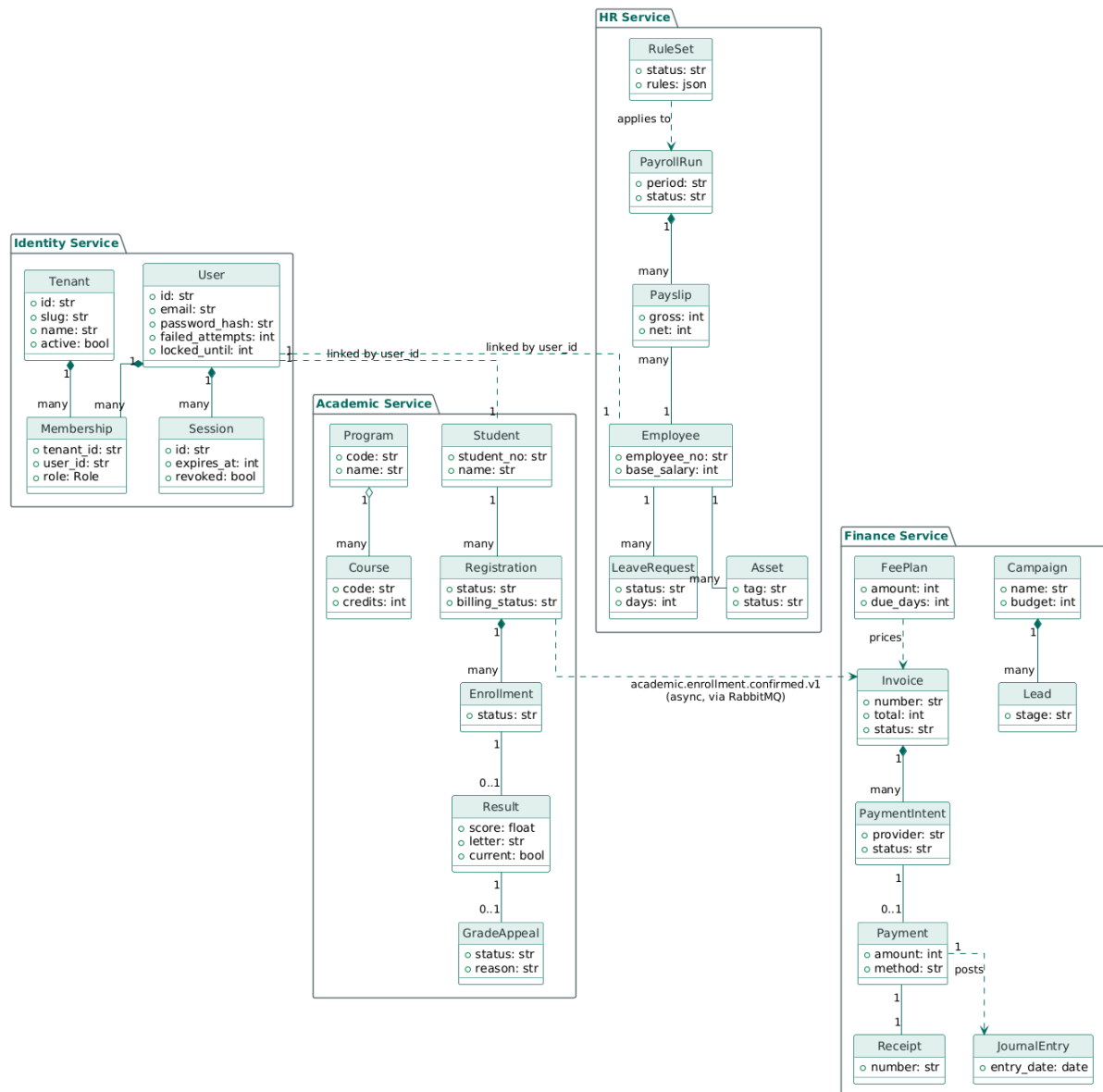


Figure 8: Domain class diagram (key entities per service, with the cross-service async link shown as a dependency)

6.3 Sequence Diagrams

6.3.1 Enrollment-to-Invoice (mandatory cross-service asynchronous workflow)

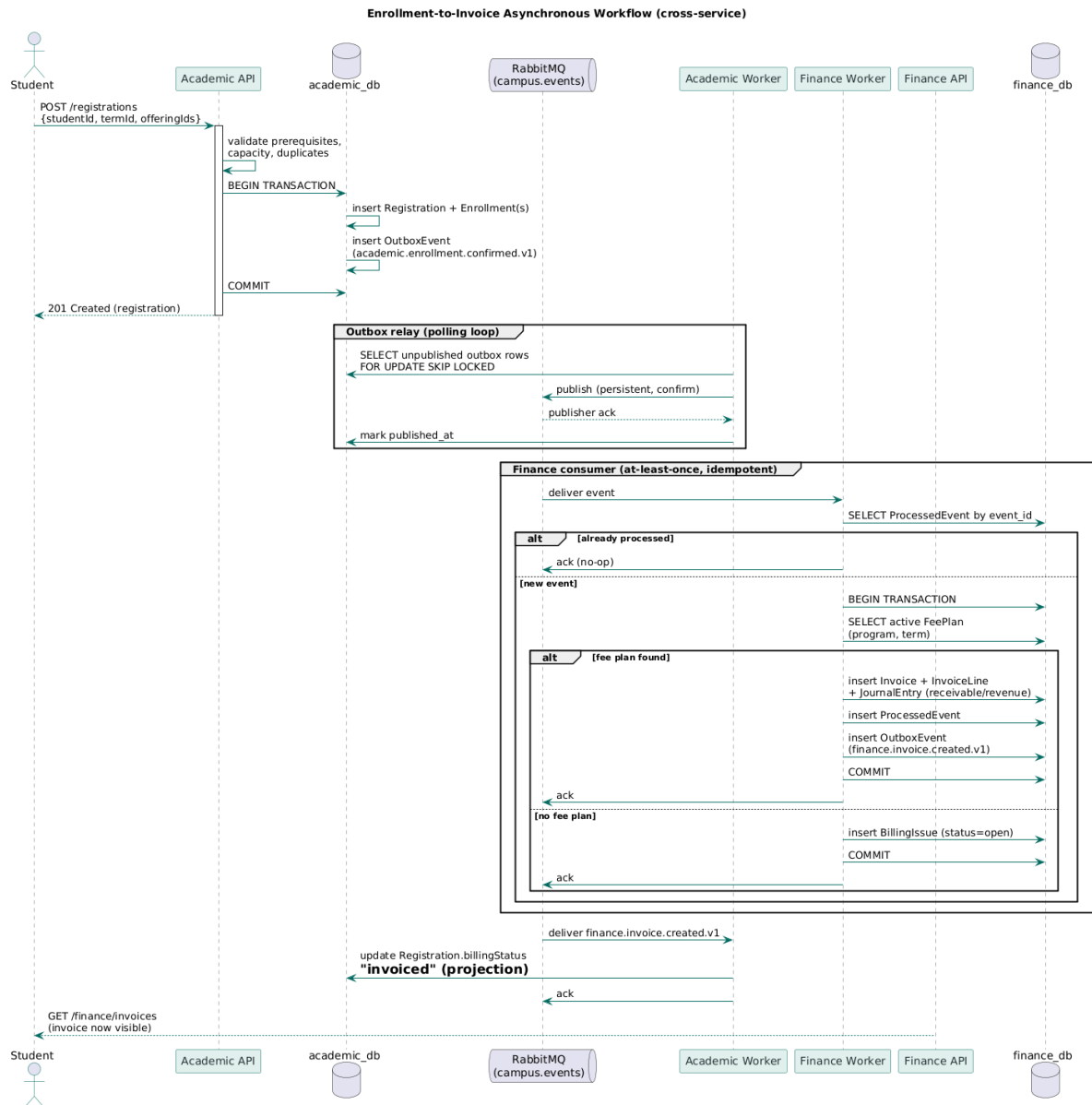


Figure 9: Sequence diagram: registration confirmation to tuition invoice via the transactional outbox and RabbitMQ

6.3.2 Mobile-Money Payment and Ledger Posting

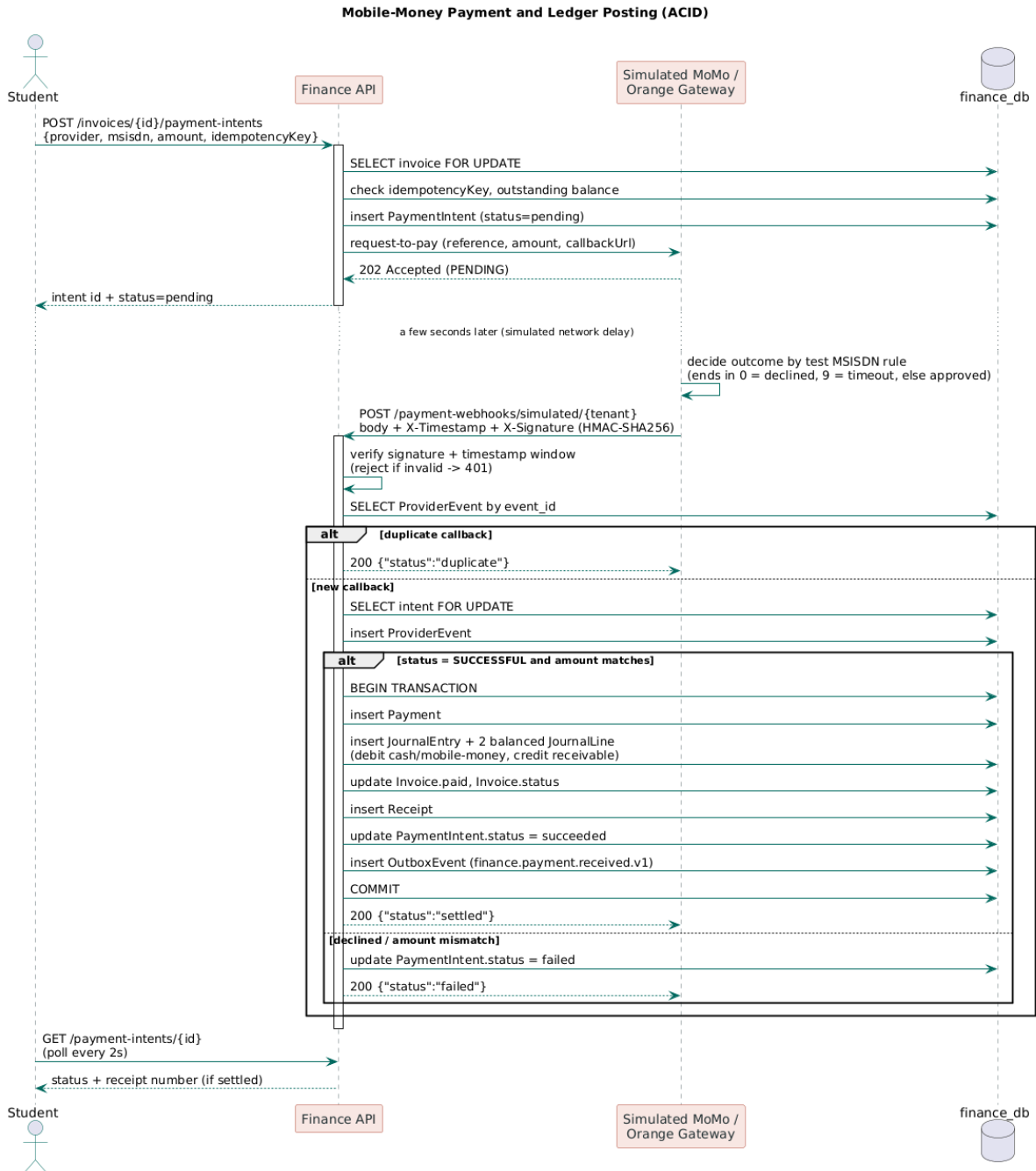


Figure 10: Sequence diagram: signed webhook callback and the ACID payment/ledger transaction

6.3.3 Authentication and Refresh-Token Rotation

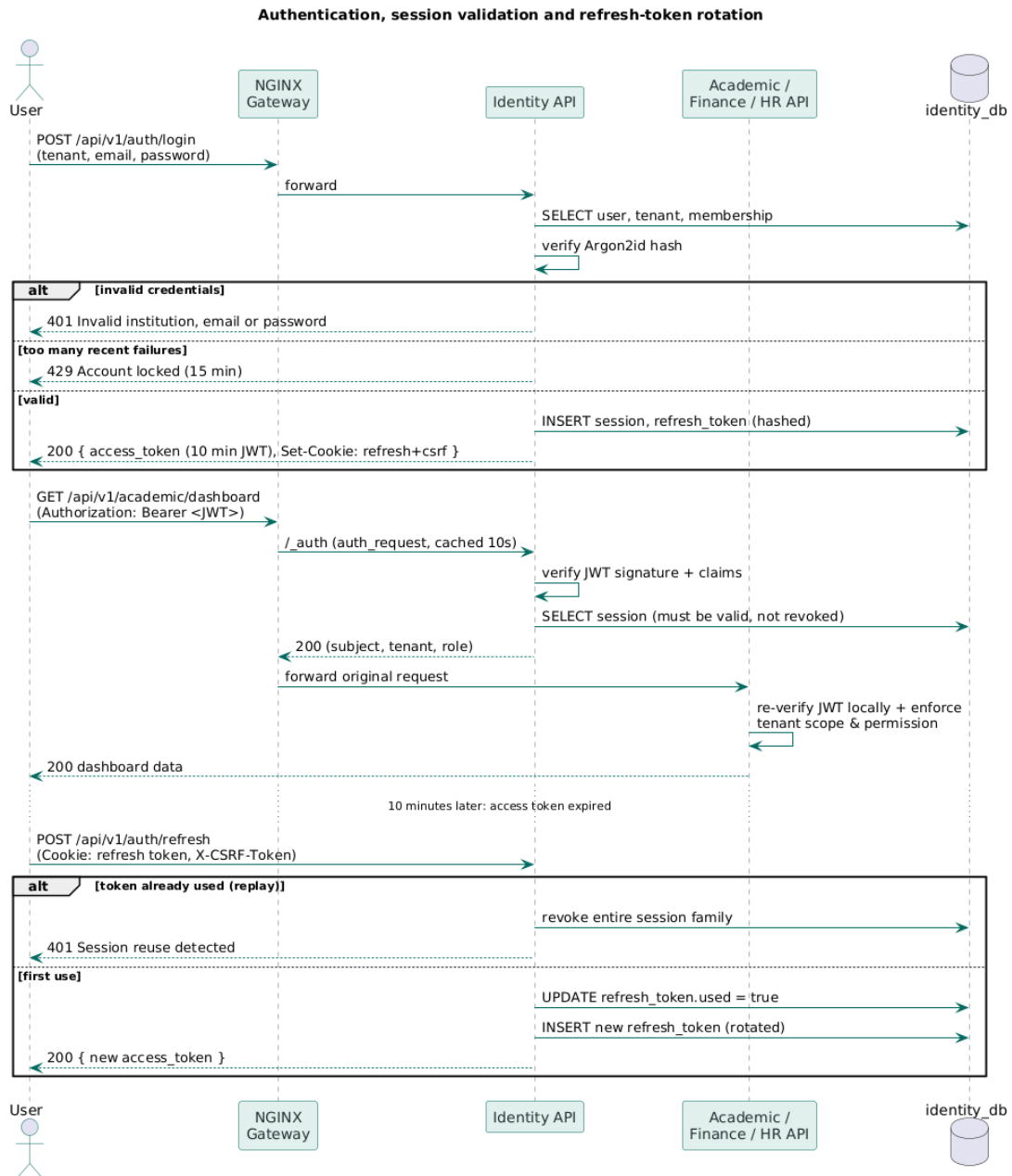


Figure 11: Sequence diagram: gateway auth delegation and refresh-token rotation with reuse detection

6.4 Deployment Diagram

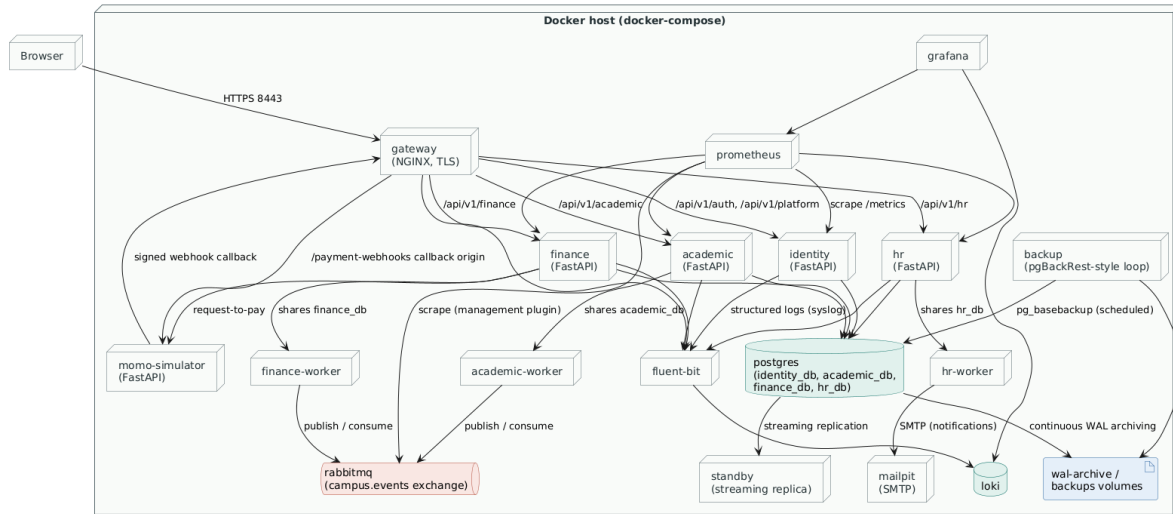


Figure 12: Deployment diagram: containers, the message broker, databases, backup/standby and observability

Diagram-to-code consistency. The four database clusters in figure 6 correspond exactly to the four `Base.metadata` declarations in `services/{identity,academic,finance,hr}/app/models.py`; the RabbitMQ exchange name `campus.events` in figures 9 and 12 matches the constant in `packages/campus_common/events.py`; and the event type `academic.enrollment.confirmed.v1` appears verbatim in `services/academic/app/domain/enrollment.py`.

7 Academic Module

7.1 Functional Requirements

- FR-1** Administrators can create, update and archive programmes and courses; archiving is used instead of deletion whenever a record is referenced by historical data, so transcripts remain valid.
- FR-2** Course prerequisites are modelled as a directed graph; adding a prerequisite that would create a cycle is rejected.
- FR-3** Registering a student for a term validates that every selected course's prerequisites are satisfied by a *published*, *passing* result – a draft or unpublished grade never satisfies a prerequisite.
- FR-4** A confirmed registration and its enrollments are recorded in one transaction together with an outbox event that ultimately triggers tuition invoicing (figure 9).
- FR-5** Instructors record assessment weights (summing to exactly 100% before publication) and per-student marks; publishing computes the weighted final score, letter grade and pass/fail outcome for every enrolled student at once.

- FR-6** Attendance is recorded per lesson per enrolled student; a late arrival counts as attended, and an excused absence is excluded from the computed rate.
- FR-7** Examinations are scheduled with automatic conflict detection across room, invigilator and any student enrolled in two overlapping offerings; a database-level advisory lock prevents two concurrent bookings from both passing the conflict check.
- FR-8** Students may appeal a published result within a configurable window (default 14 days); approval creates a new, audited result revision while preserving the original mark, never overwriting it.
- FR-9** A documented, transparent rule flags a student as at-risk in a course when their attendance falls below a configurable threshold (default 75%) or their latest two published assessment scores are both below a configurable failing mark (default 50) – explicitly a rule, not a predictive model.
- FR-10** Students and instructors can export a transcript or attendance summary as a branded PDF, generated server-side from currently published data only.

7.2 Screens

The screenshot displays the 'Examination timetable' interface within the Campus ERP system. The sidebar on the left contains navigation links for Overview, Academic (Dashboard, Course offerings, Gradebook, Class attendance, Exam timetable, Grade appeals, At-risk students), and People & Administration (My HR, Check in / out). The main content area shows the 'Examination timetable' for ICT University, with a 'Schedule exam' button. The timetable is organized by date, showing exams for Monday, Tuesday, Wednesday, and Thursday, October 19-22, 2026. Each day lists the exam name, time, room, and invigilator. For example, on Monday, SEN201 Midterm is scheduled from 09:00 to 11:00 in Hall A, invigilated by Dr. Samuel Fonkou. On Wednesday, SEN101 Midterm is scheduled from 09:00 to 11:00 in Hall B, invigilated by Dr. Samuel Fonkou, and SEN102 Midterm is scheduled from 14:00 to 16:00 in Hall B, invigilated by Dr. Grace Tabi. On Thursday, BUS201 Midterm is scheduled from 09:00 to 11:00. A 'Remove' button is visible next to the SEN201 and SEN101 exams.

Figure 13: Examination timetable with conflict-checked scheduling

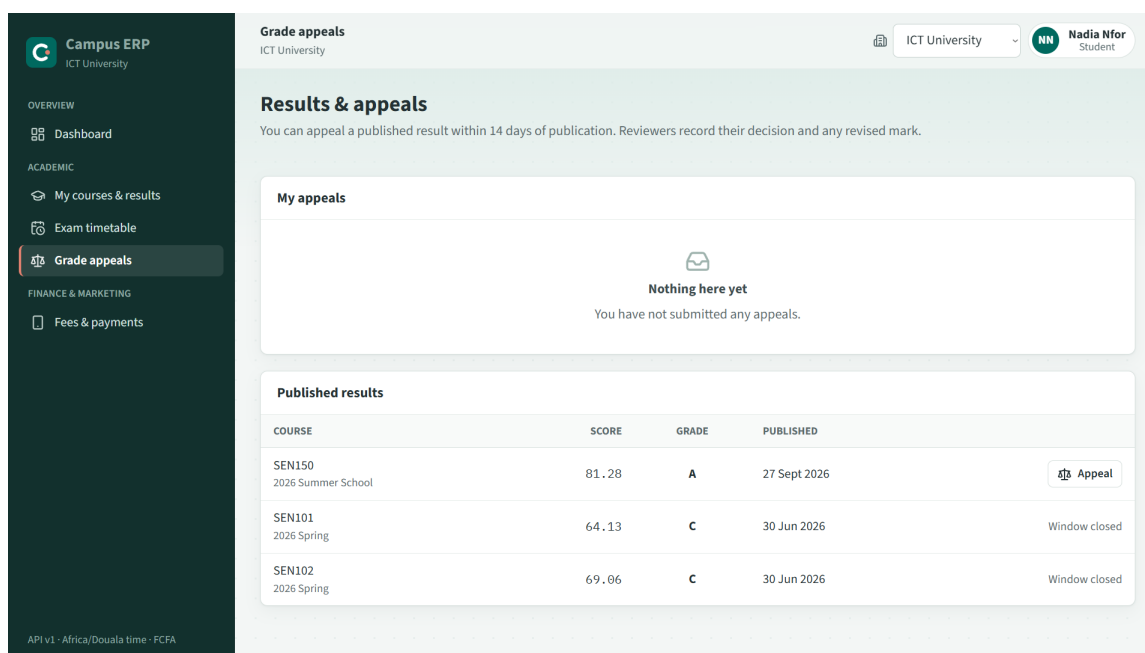


Figure 14: Student self-service: published results and the 14-day grade-appeal window

8 Marketing & Finance Module

8.1 Functional Requirements

- FR-11** A fee plan prices a programme/term combination; changing or deactivating a plan never alters invoices already issued, since every invoice stores an immutable snapshot of the price it was billed at.
- FR-12** Finance consumes `academic.enrollment.confirmed.v1` and creates exactly one invoice per registration, even under duplicate delivery; a registration whose programme/term has no active fee plan becomes a visible, retryable billing issue rather than a silent zero-value invoice.
- FR-13** Students initiate an idempotent mobile-money payment (MTN MoMo or Orange Money, simulated); the simulator emits a delayed, HMAC-signed callback supporting pending, success, failure and timeout outcomes.
- FR-14** Settlement – payment row, balanced ledger posting, invoice-balance update and digital receipt – commits as one ACID transaction (figure 10); a repeated valid callback is a safe no-op.
- FR-15** Finance officers can record a manual bank-transfer or cash payment, subject to the same outstanding-balance check.
- FR-16** Expenses follow a submitted → approved → paid workflow with segregation of duties: the approver may never be the submitter.
- FR-17** Marketing campaigns track leads through a fixed pipeline (new → contacted → qualified → applied → converted/lost); converting a lead links it to an actually invoiced student, and campaign ROI is computed only from tuition genuinely collected after conversion, divided by paid campaign spend.

FR-18 A scheduled worker automatically generates one monthly financial summary (collections, receivables, expenses, campaign metrics, all in FCFA) per tenant per closed month, catching up after downtime without producing duplicates; an authorised user may regenerate a month, which creates a new, separately retained version.

FR-19 The general ledger enforces double-entry invariants at the database layer: every posted line is a single positive debit or credit, and every entry's debits equal its credits.

8.2 Screens

The screenshot displays the 'Expenses' module in the Campus ERP system. The interface includes a sidebar with navigation options like Dashboard, Finance overview, Invoices, and Expenses. The main content area shows a table of expenses with columns for Date, Vendor, Category, Amount, and Status. The status column includes buttons for 'Approve', 'Reject', and 'Pay'. The table lists several expenses, including maintenance, supplies, marketing, and travel, with their respective amounts and statuses (Submitted, Approved, Paid).

DATE	VENDOR	CATEGORY	AMOUNT	STATUS
29 Sept 2026	Froid Service SARL Air-conditioning repair - Lab 2	Maintenance	275 000 FCFA	Submitted
28 Sept 2026	Douala Office Pro Projector lamps (x4)	Supplies	320 000 FCFA	Submitted
25 Sept 2026	Studio Mboa Campus tour video production	Marketing	400 000 FCFA	Approved
18 Sept 2026	Groupe Electro Services Generator servicing	Maintenance	450 000 FCFA	Paid
14 Sept 2026	Librairie des Peuples Office and laboratory consumables	Supplies	230 000 FCFA	Paid
12 Sept 2026	Touristique Express Recruitment fair travel - Bafoussam	Travel	185 000 FCFA	Paid

Figure 15: Expense approval workflow: submitted, to-approve, to-pay and paid queues

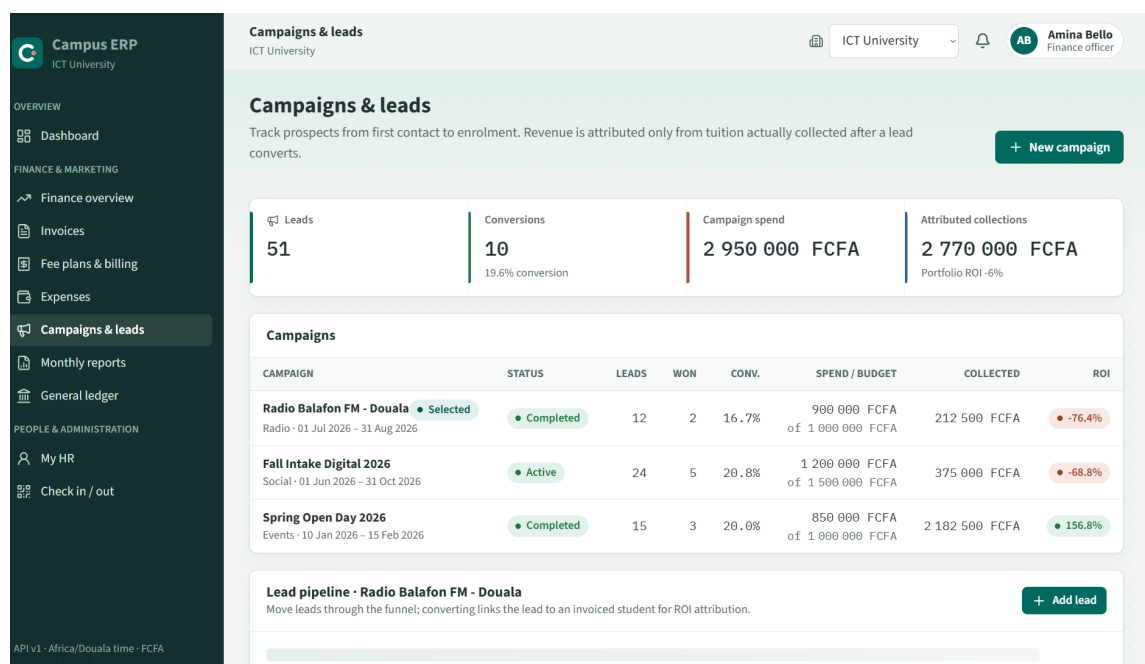


Figure 16: Marketing campaign lead pipeline and ROI, attributed only from tuition actually collected

9 Administration & Human Resource Module

9.1 Functional Requirements

- FR-20** Recruitment moves an applicant through applied → screened → interview → offered → hired/rejected; hiring atomically creates the employee record and initial leave entitlements.
- FR-21** Payroll is computed from an editable, effective-dated, source-cited *rule set* covering CNPS pension, family-allowance and work-injury contributions (employee and employer shares, with a monthly contribution ceiling) and the IRPP/PAYE progressive scale with CAC, following a professional-expense allowance and an annual abatement.
- FR-22** **Statutory-rate governance (explicit, honest statement).** A rule set is created in **draft** status and cannot be used to run payroll until an HR officer records, in an approval note, which official publication they verified each rate against, and explicitly approves it. This governance gate exists because payroll tax law changes over time and must never be hard-coded as if permanently correct; the sources actually consulted while preparing this submission are listed in Appendix B, together with an explicit note on which figures still require the officer's sign-off before the system is used for anything beyond a classroom demonstration.
- FR-23** A finalized payroll run is immutable and unique per employee per period; each payslip itemises every employee deduction and every employer contribution separately, and finalisation notifies every paid employee with a login.
- FR-24** Employee attendance uses a kiosk-displayed, HMAC-signed, 60-second expiring QR challenge – not a static, reusable employee identifier, and not biometric hardware (an explicitly permitted documented substitute). Scanning or typing the underlying code checks the employee in, and a second scan the same day checks them out.

- FR-25** Leave requests are checked against the employee's remaining entitlement (including other pending requests) before submission is accepted; approval and balance decrement happen under a database row lock so two concurrent approvals cannot overdraw the entitlement; an employee can never approve their own request.
- FR-26** Performance reviews record goals, a 1–5 rating and reviewer comments; a completed review is locked, and the employee is notified.
- FR-27** Physical assets (tag, category, condition, assignment) and consumable inventory (an append-only stock-movement ledger) are tracked with concurrency-safe checks that block double assignment and negative stock; low stock triggers an in-app notification to HR.
- FR-28** Critical events (leave decisions, low stock, completed payroll) are persisted as in-app notifications and additionally queued for email delivery; a transient mail-server outage never blocks or reverses the underlying business decision.

9.2 Screens

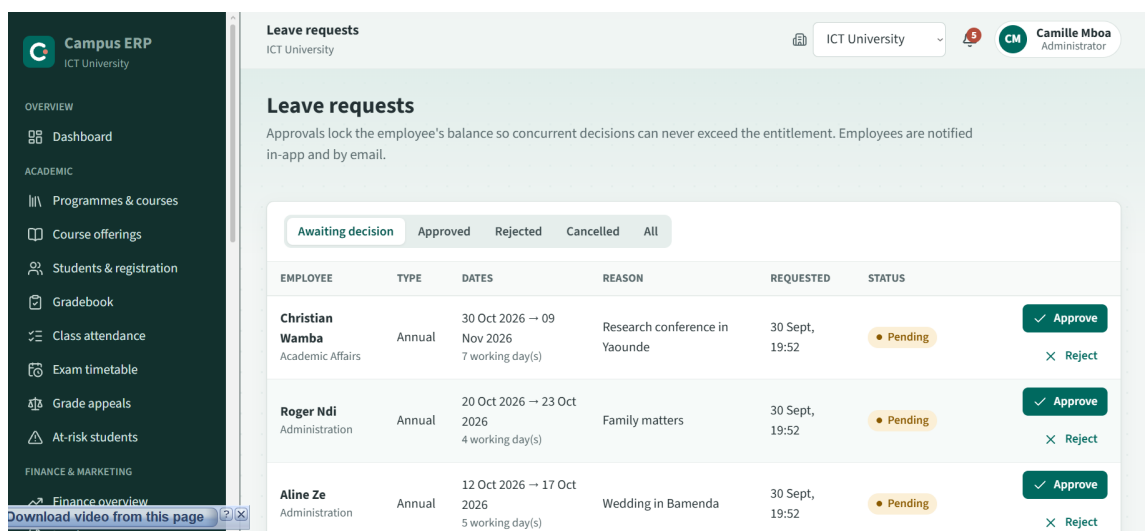


Figure 17: Leave request approval screen: an employee cannot approve their own request

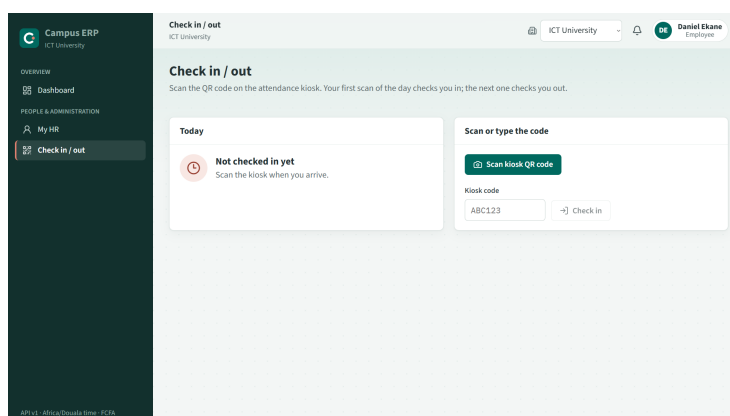


Figure 18: Employee self-service QR check-in screen

10 Non-Functional Evidence, Testing and DevOps

10.1 Performance and Load Testing

The defined workload is 50 concurrent virtual users against a read-heavy mix of dashboard, list and detail endpoints for five minutes after a warm-up period, using k6. The proposed acceptance targets are a 95th-percentile response time under 500 ms for read endpoints and under 1000 ms for write endpoints, with an unexpected-error rate below 1%. Login and report-export endpoints are measured separately from the main workload, since they are deliberately rate-limited at the gateway. table 8 is the template used to record the actual run; it should be completed with real numbers and hardware details from `tests/load/` before final submission.

Table 8: Load-test result template (complete with measured values)

Workload	VUs	p95 (ms)	Errors (%)	Duration
Dashboard reads	50	–	–	5 min
Registration write	10	–	–	2 min
Payment intent + callback	10	–	–	2 min

10.2 Scalability Strategy

API instances are stateless with respect to authentication (JWT verification uses only the public key, held by every instance), so the gateway can load balance multiple replicas of any domain service without sticky sessions. Identity’s session and refresh-token state lives centrally in `identity_db`, which remains the single source of truth for session revocation across replicas. Consumers use a bounded prefetch count so that adding worker replicas increases throughput without unbounded memory growth, and every consumer transaction is idempotent, so redelivery after a crash or a rebalance cannot duplicate side effects. figure 12’s Compose topology is the current single-node profile; horizontally scaling any one service is, by this design, a configuration change (replica count) rather than a code change.

10.3 Testing and Quality Assurance

Each service owns a `tests/` suite of unit tests (pure domain-function logic, e.g. conflict detection, at-risk evaluation, payroll calculation) and integration tests (through the FastAPI test client, covering full HTTP request/response cycles, tenant isolation, and concurrency/rollback scenarios). Coverage is measured on each service’s `app.domain` package specifically, so that trivial routing code cannot inflate the percentage; the CI pipeline fails the build below 70%. table 9 reports the coverage achieved at the time of writing (see `docs/evidence/coverage.txt` for the full `pytest-cov` report).

Table 9: Automated test coverage by service (core domain logic)

Service	Tests	Domain coverage	Status
Identity	15	96%	Passing
Academic	21	~89%	Passing
Finance	8	~88%	Passing
HR	11	~88%	Passing

A written test plan maps each functional requirement (Appendix A) to the automated test

that verifies it. Open and resolved issues are tracked on a GitHub Issues board (linked from the repository README); screenshots or an export of that board should be attached as evidence before submission.

10.4 DevOps: Containerisation and CI/CD

Every service (and its worker) has its own multi-stage Dockerfile, running as a non-root user, with a container HEALTHCHECK against its `/health/ready` endpoint. `docker-compose.yml` brings up the full system – gateway, four services, three workers, the mobile-money simulator, PostgreSQL, RabbitMQ, Mailpit and the observability stack – with one command, gated by explicit `depends_on` health and completion conditions so that migrations run before the API starts and the API is healthy before the gateway routes to it. The GitHub Actions pipeline (`.github/workflows/ci.yml`) runs on every pull request and on every push to `main`: it lints and tests each Python service independently with a coverage gate, builds and type-checks the frontend, and – on `main` only – builds every service’s container image, verifying the Dockerfiles are still valid without publishing to a registry.

10.5 Deployment Guide (Summary)

A third party can run the system from a clean machine with Docker Desktop installed by cloning the repository, running `python scripts/bootstrap.py` (generates `.env` and local JWT signing keys), then `docker compose up -build`. The full step-by-step guide, including the non-Docker quick-start (`run.bat`) used for this report’s screenshots, is kept in the repository README and is not duplicated here to preserve the page budget.

11 Limitations, Future Work and Academic Integrity

11.1 Known Limitations

- **Payment gateway.** MTN Mobile Money and Orange Money are represented by a disclosed simulator that mimics their request-to-pay and signed-callback contract; no real money moves and no production credentials are held. Replacing the simulator with a certified adapter is future work and is isolated behind the `PaymentProvider` interface.
- **Statutory payroll rates.** The CNPS/IRPP rule set must be reviewed and explicitly approved by an HR officer against the current official publications before it is relied upon for anything beyond a classroom demonstration; see section 9 and Appendix B.
- **Biometric attendance.** A signed, time-boxed QR challenge is used instead of biometric hardware, as explicitly permitted by the project brief.
- **High availability.** A streaming standby is included for the backup/recovery story, but automatic failover, multi-region deployment and Kubernetes manifests were not attempted (the brief lists Kubernetes as optional bonus work).
- **Multi-factor authentication.** Listed as optional bonus work in the brief; not implemented in this submission.
- **Deployment scale.** The demonstrated deployment is a single Docker host; the scalability strategy in section 10 is validated conceptually (stateless services, idempotent consumers) rather than against a multi-node cluster.

11.2 Future Work

Priority candidates for continued development are: a certified mobile-money adapter; a second, independently verified payroll rule-set review by a qualified accountant; Kubernetes manifests for the optional bonus marks; and extending the load-testing workload to the write-heavy payroll-finalisation and monthly-report-generation paths.

11.3 Academic Integrity and AI-Assistance Disclosure

This project’s codebase, infrastructure configuration and this report were developed with the assistance of an AI pair-programming tool (GitHub Copilot), used for scaffolding boilerplate, refactoring, and drafting prose and diagram source, under the direction and review of the author, **CHII BENEDICTA YIESE (ICTU20251667)**. Every architectural decision, business rule and verification claim in this document reflects a choice made and checked by the author, and the author is able to explain and, if asked, modify any part of the delivered codebase during the individual defense. This disclosure is made in accordance with the module’s academic-integrity policy, which requires AI-generated or reused material to be explicitly cited and explained at the start of the defense.

Reminder for submission. Attach the signed individual contribution statement and the signed academic-integrity declaration as separate documents, per the brief – they are deliberately *not* included inside this report’s page count.

A Requirement Traceability Matrix

ID	Requirement	Owning module	Verified by
FR-1	Prerequisite enforcement on registration	Academic	<code>test_academic.py::test_prere</code>
FR-2	Exam conflict detection (room/invigilator/student)	Academic	<code>test_academic.py::test_exam</code>
FR-3	Grade appeal preserves original mark	Academic	<code>test_academic.py::test_prere</code>
FR-4	At-risk rule (attendance / consecutive failures)	Academic	<code>test_academic.py::test_atten</code>
FR-5	Exactly-once invoicing under duplicate events	Finance	<code>test_finance.py::test_enroll</code>
FR-6	ACID payment + ledger posting, rollback on fault	Finance	<code>test_finance.py::test_paymen</code>
FR-7	Campaign ROI attribution	Finance	<code>test_finance.py::test_expens</code>
FR-8	Monthly report scheduler, no duplicates	Finance	<code>test_finance.py::test_monthl</code>
FR-9	Payroll blocked until rule set approved	HR	<code>test_hr.py::test_payroll_req</code>
FR-10	QR challenge expiry and tenant binding	HR	<code>test_hr.py::test_qr_attendan</code>
FR-11	Leave balance cannot be overdrawn (concurrency)	HR	<code>test_hr.py::test_leave_workf</code>
FR-12	Cross-tenant access denied	Identity + all	<code>test_academic.py::test_tenan</code> equivalents in other services
FR-13	Refresh-token rotation and reuse detection	Identity	<code>test_sessions.py::test_refre</code>
FR-14	Account lockout after failed attempts	Identity	<code>test_sessions.py::test_locko</code>

B Cited Sources and Third-Party Material

B.1 Statutory Payroll Rate Sources

The draft CNPS/IRPP rule set shipped with the demonstration data cites the following official sources; an HR officer must verify each rate against the current version of these publications and record the verification before approving the rule set for use (section 9):

- Direction Générale des Impôts, *Code Général des Impôts*, IRPP salary provisions (professional-expense allowance, annual abatement, progressive scale, CAC) – <https://www.impots.cm/fr/documentation>
- Direction Générale des Impôts, *Circulaire portant instructions relatives à l'application de la Loi de Finances 2026* – <https://www.impots.cm/fr/actualites/circulaire-lf-2026>
- Caisse Nationale de Prévoyance Sociale (CNPS) contribution schedule (pension, family allowance, work-injury; contribution ceiling) – <https://www.cnps.cm/>

B.2 Third-Party Software

Campus ERP is built on open-source software including FastAPI, SQLAlchemy, Pydantic, PyJWT, Argon2-cffi, pika, ReportLab, qrcode, React, Vite, TanStack Query, Recharts, Lucide, PostgreSQL, RabbitMQ, NGINX, Prometheus, Grafana, Loki and Fluent Bit, each used under its respective open-source licence. No proprietary third-party code was reused.

B.3 AI-Assistance Disclosure

See section 11 for the full academic-integrity disclosure regarding AI pair-programming assistance used while developing this project and drafting this report.